

C# + WPF (.NET Framework 4.8) + MySQL (Visual Studio Community 2026)

Results

Full worked software по модулям 1-4:

- база данных MySQL в ЗНФ;
 - импорт исходных данных из `xlsx` (через `csv`) в `phpMyAdmin`;
 - окно входа и роли (гость, клиент, менеджер, администратор);
 - список товаров с подсветкой скидок/остатков, поиском, фильтром и сортировкой по ролям;
 - форма добавления/редактирования товара для администратора;
 - окно заказов и форма добавления/редактирования заказа;
 - ER-диаграмма БД в PDF, SQL-скрипт БД, `algorithm_gost.pdf`, `report_screenshots.docx`;
 - исполняемые файлы из `bin\Release`;
 - локальный `git`-репозиторий с зафиксированной итоговой версией проекта.
-

Step 1 Create рабочую структуру проекта

Create папку проекта и подкаталоги для ресурсов и документов.

Commands/code

```
cd C:\
mkdir shoe_store_2026_pu_csharp
cd shoe_store_2026_pu_csharp
mkdir resources
mkdir resources\photos
mkdir docs
mkdir sql
mkdir screenshots
```

Copy from папки Вариант1(2026)\Данные\Модуль 1\Прил_2_03_КОД 09.02.07-2-2026-M1\import в `C:\shoe_store_2026_pu_csharp\resources`:

- `Icon.ico`
- `Icon.png`
- `picture.png`
- `Tovar.xlsx`
- `user_import.xlsx`
- Пункты выдачи_import.xlsx
- Заказ_import.xlsx

Copy folder photos в:

- `C:\shoe_store_2026_pu_csharp\resources\photos`
-

Step 2 Run MySQL + phpMyAdmin через Docker Compose

Create `docker-compose.yml` и запустите контейнеры. Альтернативно (без Docker) можно использовать XAMPP, Open Server Panel или локально установленный MySQL Server.

Commands/code

Create файл `C:\shoe_store_2026_pu_csharp\docker-compose.yml`:

```
services:
```

```
mysql:
  image: mysql:8.4
  container_name: shoe2026_pu_mysql
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: shoe2026_pu
    MYSQL_USER: demo
    MYSQL_PASSWORD: demo
  ports:
    - "3306:3306"
  volumes:
    - mysql_data:/var/lib/mysql

phpmyadmin:
  image: phpmyadmin:latest
  container_name: shoe2026_pu_phpmyadmin
  restart: unless-stopped
  environment:
    PMA_HOST: mysql
    PMA_PORT: 3306
    PMA_USER: root
    PMA_PASSWORD: root
  ports:
    - "8081:80"
  depends_on:
    - mysql

volumes:
  mysql_data:
```

Run:

```
cd C:\shoe_store_2026_pu_csharp
docker compose up -d
docker compose ps
```

Open phpMyAdmin:

- при Docker: <http://localhost:8081>, логин root, пароль root;
- при XAMPP/Open Server Panel/локальном MySQL Server: your локальный адрес phpMyAdmin (часто <http://localhost/phpmyadmin>).

Step 3 Create схему БД (модули 1-4)

Create рабочие таблицы, связи и raw-таблицы для импорта.

Commands/code

1. Если БД shoe2026_pu еще не создана: вкладка Базы данных -> имя shoe2026_pu -> сравнение utf8mb4_unicode_ci -> Создать.
2. Select БД shoe2026_pu.
3. Open вкладку SQL и выполните:

```
USE shoe2026_pu;

SET NAMES utf8mb4;

CREATE TABLE roles (
  role_id TINYINT UNSIGNED PRIMARY KEY,
  role_name VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE users (
  user_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  role_id TINYINT UNSIGNED NOT NULL,
  full_name VARCHAR(200) NOT NULL,
  login VARCHAR(120) NOT NULL UNIQUE,
  password_plain VARCHAR(120) NOT NULL,
  CONSTRAINT fk_users_roles
    FOREIGN KEY (role_id) REFERENCES roles(role_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE categories (
  category_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE manufacturers (
  manufacturer_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```

CREATE TABLE suppliers (
  supplier_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE products (
  product_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  article VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(200) NOT NULL,
  unit_name VARCHAR(20) NOT NULL,
  price DECIMAL(12,2) NOT NULL CHECK (price >= 0),
  supplier_id INT UNSIGNED NOT NULL,
  manufacturer_id INT UNSIGNED NOT NULL,
  category_id INT UNSIGNED NOT NULL,
  discount_percent DECIMAL(5,2) NOT NULL CHECK (discount_percent >= 0),
  stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
  description text TEXT NULL,
  photo_file VARCHAR(255) NULL,
  CONSTRAINT fk_products_suppliers
    FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_products_manufacturers
    FOREIGN KEY (manufacturer_id) REFERENCES manufacturers(manufacturer_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_products_categories
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE pickup_points (
  pickup_point_id INT UNSIGNED PRIMARY KEY,
  address_text VARCHAR(255) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE order_statuses (
  status_id TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  status_name VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE orders (
  order_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_number INT UNSIGNED NOT NULL UNIQUE,
  article_text VARCHAR(255) NOT NULL,
  order_date DATE NULL,
  delivery_date DATE NULL,
  pickup_point_id INT UNSIGNED NOT NULL,
  client_user_id INT UNSIGNED NULL,
  pickup_code INT UNSIGNED NOT NULL,
  status_id TINYINT UNSIGNED NOT NULL,
  CONSTRAINT fk_orders_pickup_points
    FOREIGN KEY (pickup_point_id) REFERENCES pickup_points(pickup_point_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_orders_users
    FOREIGN KEY (client_user_id) REFERENCES users(user_id)
    ON UPDATE CASCADE ON DELETE SET NULL,
  CONSTRAINT fk_orders_statuses
    FOREIGN KEY (status_id) REFERENCES order_statuses(status_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE order_items (
  order_item_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_id INT UNSIGNED NOT NULL,
  product_id INT UNSIGNED NOT NULL,
  quantity INT NOT NULL CHECK (quantity > 0),
  CONSTRAINT fk_order_items_order
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_order_items_product
    FOREIGN KEY (product_id) REFERENCES products(product_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT uk_order_product UNIQUE (order_id, product_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_products_supplier ON products(supplier_id);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_orders_status ON orders(status_id);
CREATE INDEX idx_order_items_product ON order_items(product_id);

CREATE TABLE users_import_raw (
  role_name VARCHAR(100),
  full_name VARCHAR(200),
  login_text VARCHAR(120),
  password_text VARCHAR(120)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE products_import_raw (
  article_text VARCHAR(60),
  name_text VARCHAR(200),
  unit_text VARCHAR(20),
  price_text VARCHAR(40),
  supplier_text VARCHAR(120),
  manufacturer_text VARCHAR(120),
  category_text VARCHAR(120),
  discount_text VARCHAR(40),

```

```

stock_text VARCHAR(40),
description_text TEXT,
photo_text VARCHAR(120)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE pickup_points_import_raw (
  raw_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  address_text VARCHAR(255)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE orders_import_raw (
  raw_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  order_number_text VARCHAR(40),
  articles_text VARCHAR(255),
  order_date_text VARCHAR(40),
  delivery_date_text VARCHAR(40),
  pickup_point_text VARCHAR(40),
  client_fio_text VARCHAR(200),
  pickup_code_text VARCHAR(40),
  status_text VARCHAR(80)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Step 4 Prepare CSV из исходных XLSX (Через Excel)

Open every xlsx в Excel, prepare строки и save CSV UTF-8.

Commands/code

Сначала prepare xlsx, затем save csv:

1. В user_import.xlsx, Товар.xlsx, Заказ_import.xlsx delete первую строку с заголовками.
2. В Пункты выдачи_import.xlsx первую строку не delete (это данные).
3. В Заказ_import.xlsx delete пустые столбцы I, J, K, L.
4. Save как CSV UTF-8:
 - user_import.csv
 - Товар.csv
 - Пункты выдачи_import.csv
 - Заказ_import.csv
5. Open every csv в Блокноте и delete пустые строки в конце файла (если есть).
6. Check символ-разделитель столбцов в файле (; или ,), этот символ use на шаге импорта.

Step 5 Import данные через UI phpMyAdmin

Import CSV в raw-таблицы.

Commands/code

В phpMyAdmin:

1. Таблица users_import_raw -> Импорт -> user_import.csv -> Названия столбцов:
role_name, full_name, login_text, password_text
2. Таблица products_import_raw -> Импорт -> Товар.csv -> Названия столбцов:
article_text, name_text, unit_text, price_text, supplier_text, manufacturer_text, category_text, discount_text, stock_text, description_text, photo_text
3. Таблица pickup_points_import_raw -> Импорт -> Пункты выдачи_import.csv -> Названия столбцов:
address_text
4. Таблица orders_import_raw -> Импорт -> Заказ_import.csv -> Названия столбцов:
order_number_text, articles_text, order_date_text, delivery_date_text, pickup_point_text, client_fio_text, pickup_code_text, status_text

For every импорта set:

- Формат = CSV
- Кодировка файла = utf-8
- Разделитель полей = символ your CSV (; или ,)
- Значения полей обрамлены = "
- Символ экранирования = \"
- Разделитель строк = auto
- Названия столбцов = as above

Step 5.1 Transfer данные from raw in боевые таблицы

Fill рабочие таблицы и связи.

Commands/code

Во вкладке SQL execute:

```
INSERT INTO roles (role_id, role_name) VALUES
(1, 'Гость'),
(2, 'Авторизованный клиент'),
(3, 'Менеджер'),
(4, 'Администратор');

INSERT INTO users (role_id, full_name, login, password_plain)
SELECT
CASE
WHEN role_name LIKE '%Администратор%' THEN 4
WHEN role_name LIKE '%Менеджер%' THEN 3
ELSE 2
END AS role_id,
TRIM(full_name),
TRIM(login_text),
TRIM(REPLACE(password_text, '\r', ''))
FROM users_import_raw
WHERE TRIM(login_text) <> '';

INSERT INTO categories (name)
SELECT DISTINCT TRIM(category_text)
FROM products_import_raw
WHERE TRIM(category_text) <> '';

INSERT INTO manufacturers (name)
SELECT DISTINCT TRIM(manufacturer_text)
FROM products_import_raw
WHERE TRIM(manufacturer_text) <> '';

INSERT INTO suppliers (name)
SELECT DISTINCT TRIM(supplier_text)
FROM products_import_raw
WHERE TRIM(supplier_text) <> '';

INSERT INTO products (
article, name, unit_name, price,
supplier_id, manufacturer_id, category_id,
discount_percent, stock_quantity, description_text, photo_file
)
SELECT
TRIM(p.article_text),
TRIM(p.name_text),
TRIM(p.unit_text),
CAST(REPLACE(TRIM(p.price_text), ',', '.') AS DECIMAL(12,2)),
s.supplier_id,
m.manufacturer_id,
c.category_id,
CAST(REPLACE(TRIM(p.discount_text), ',', '.') AS DECIMAL(5,2)),
CAST(TRIM(p.stock_text) AS UNSIGNED),
NULLIF(TRIM(p.description_text), ''),
NULLIF(TRIM(REPLACE(p.photo_text, '\r', '')), '')
FROM products_import_raw p
JOIN suppliers s ON s.name = TRIM(p.supplier_text)
JOIN manufacturers m ON m.name = TRIM(p.manufacturer_text)
JOIN categories c ON c.name = TRIM(p.category_text)
WHERE TRIM(p.article_text) <> '';

INSERT INTO pickup_points (pickup_point_id, address_text)
SELECT raw_id, TRIM(REPLACE(address_text, '\r', ''))
FROM pickup_points_import_raw
WHERE TRIM(REPLACE(address_text, '\r', '')) <> ''
ORDER BY raw_id;
```

```

INSERT INTO order_statuses (status_name)
SELECT DISTINCT TRIM(REPLACE(status_text, '\r', ''))
FROM orders_import_raw
WHERE TRIM(REPLACE(status_text, '\r', '')) <> '';

INSERT INTO orders (
    order_number, article_text, order_date, delivery_date,
    pickup_point_id, client_user_id, pickup_code, status_id
)
SELECT
    CAST(TRIM(o.order_number_text) AS UNSIGNED),
    TRIM(o.articles_text),
    STR_TO_DATE(TRIM(o.order_date_text), '%d.%m.%Y'),
    STR_TO_DATE(TRIM(o.delivery_date_text), '%d.%m.%Y'),
    CAST(TRIM(o.pickup_point_text) AS UNSIGNED),
    u.user_id,
    CAST(TRIM(o.pickup_code_text) AS UNSIGNED),
    st.status_id
FROM orders_import_raw o
LEFT JOIN users u
    ON u.full_name = TRIM(o.client_fio_text)
JOIN order_statuses st
    ON st.status_name = REPLACE(TRIM(o.status_text), '\r', '')
WHERE TRIM(o.order_number_text) <> '';

INSERT INTO order_items (order_id, product_id, quantity)
SELECT
    o.order_id,
    pr.product_id,
    CAST(TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(o.article_text, ',', 2), ',', -1)) AS UNSIGNED) AS qty
FROM orders o
JOIN products pr
    ON pr.article = TRIM(SUBSTRING_INDEX(o.article_text, ',', 1))
WHERE TRIM(o.article_text) <> ''

UNION ALL

SELECT
    o.order_id,
    pr.product_id,
    CAST(TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(o.article_text, ',', 4), ',', -1)) AS UNSIGNED) AS qty
FROM orders o
JOIN products pr
    ON pr.article = TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(o.article_text, ',', 3), ',', -1))
WHERE TRIM(o.article_text) <> '';

```

Step 6 Execute контрольные SQL-проверки

Check, что данные загружены полностью.

Commands/code

```

USE shoe2026_pu;

SELECT 'roles' AS table_name, COUNT(*) AS cnt FROM roles
UNION ALL SELECT 'users', COUNT(*) FROM users
UNION ALL SELECT 'categories', COUNT(*) FROM categories
UNION ALL SELECT 'manufacturers', COUNT(*) FROM manufacturers
UNION ALL SELECT 'suppliers', COUNT(*) FROM suppliers
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'pickup_points', COUNT(*) FROM pickup_points
UNION ALL SELECT 'order_statuses', COUNT(*) FROM order_statuses
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'order_items', COUNT(*) FROM order_items;

```

Должно получиться:

- roles = 4
- users = 10
- categories = 2
- manufacturers = 6
- suppliers = 2
- products = 30
- pickup_points = 36
- order_statuses = 2
- orders = 10
- order_items = 20

Для входа в приложение:

```
SELECT
    u.login,
    TRIM(REPLACE(u.password_plain, CHAR(13), '')) AS password_plain,
    u.full_name,
    r.role_name
FROM users u
JOIN roles r ON r.role_id = u.role_id
ORDER BY r.role_id, u.full_name;
```

Step 7 Create WPF-проект, set пакет MySQL и add ресурсы

Create проект в Visual Studio Community 2026, set MySql.Data и add файлы ресурсов в проект.

Commands/code

1. Open Visual Studio Community 2026.
2. Click Создать проект.
3. Select шаблон Приложение WPF (.NET Framework).
4. Имя проекта: ShoeStore2026PUApp.
5. Путь: C:\shoe_store_2026_pu_csharp.
6. Framework: .NET Framework 4.8.
7. Click Создать.
8. Меню Проект -> Управление пакетами NuGet... -> вкладка Обзор -> установите MySql.Data версии 8.4.0.
9. Если окно не see: Вид -> Обзорщик решений.
10. Define фактическую папку проекта: это папка, где лежит App.xaml.
11. Из C:\shoe_store_2026_pu_csharp\resources сору в папку проекта (где App.xaml) папку resources с вложенной photos.
12. В Обзорщик решений click Показать все файлы.
13. Если папка resources и файлы серые: ПКМ -> Включить в проект.
14. Для Icon.ico, Icon.png, picture.png и всех jpg в свойствах (F4) set:
 - Действие при сборке = Resource.

Step 8 Create Db.cs и check подключение к MySQL

Take it out строку подключения в отдельный класс и check соединение через код.

Commands/code

Create файл Db.cs:

- Обзорщик решений -> RMB по проекту -> Добавить -> Класс... -> имя Db.cs.

```
using MySql.Data.MySqlClient;

namespace ShoeStore2026PUApp
{
    internal static class Db
    {
        public const string ConnectionString =
            "Server=127.0.0.1;Port=3306;Database=shoe2026_pu;Uid=demo;Pwd=demo;Charset=utf8mb4;Allow Zero
            ↳ Datetime=True;Convert Zero Datetime=True;";

        public static MySqlConnection GetConnection()
        {
            return new MySqlConnection(ConnectionString);
        }
    }
}
```

Если use XAMPP/Open Server Panel/локальный MySQL Server, change параметры подключения под your локальную установку (часто root и пустой пароль).

Open App.xaml.cs и временно replace содержимое на:

```
using System.Windows;

namespace ShoeStore2026PUApp
{
    public partial class App : Application
    {
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            try
            {
                using (var conn = Db.GetConnection())
                {
                    conn.Open();
                }
                MessageBox.Show("OK: подключение к MySQL успешно.", "Проверка БД",
                    MessageBoxButton.OK, MessageBoxImage.Information);
            }
            catch (System.Exception ex)
            {
                MessageBox.Show("Ошибка подключения к MySQL:\n" + ex.Message, "Проверка БД",
                    MessageBoxButton.OK, MessageBoxImage.Error);
            }
        }
    }
}
```

Execute F5. После checking, return App.xaml.cs в исходное состояние.

Step 9 Create Models.cs

Create модели для ролей, товаров и заказов.

Commands/code

Create файл Models.cs:

- Обзоратель решений -> RMB по проекту -> Добавить -> Класс... -> имя Models.cs.

```
using System;
using System.Windows.Media;
using System.Windows.Media.Imaging;

namespace ShoeStore2026PUApp
{
    public class UserInfo
    {
        public int? UserId { get; set; }
        public string FullName { get; set; }
        public string RoleName { get; set; }
    }

    public class LookupItem
    {
        public int Id { get; set; }
        public string Name { get; set; }

        public override string ToString()
        {
            return Name;
        }
    }

    public class ProductRow
    {
        public int ProductId { get; set; }
        public string Article { get; set; }
        public string Name { get; set; }
        public string Category { get; set; }
        public string Description { get; set; }
        public string Manufacturer { get; set; }
        public string Supplier { get; set; }
        public decimal Price { get; set; }
        public decimal DiscountPercent { get; set; }
        public decimal FinalPrice { get; set; }
        public string UnitName { get; set; }
        public int StockQuantity { get; set; }
        public bool HasDiscount { get; set; }
        public string PhotoFile { get; set; }
        public BitmapImage PhotoImage { get; set; }
    }
}
```



```

    public Brush RowBrush
    {
        get
        {
            if (StockQuantity == 0)
                return new SolidColorBrush((Color)ColorConverter.ConvertFromString("#ADD8E6"));
            if (DiscountPercent > 15m)
                return new SolidColorBrush((Color)ColorConverter.ConvertFromString("#2E8B57"));
            return new SolidColorBrush((Color)ColorConverter.ConvertFromString("FFFFFF"));
        }
    }

    public Brush RowForeground
    {
        get
        {
            return new SolidColorBrush((Color)ColorConverter.ConvertFromString("#11827"));
        }
    }
}

public class ProductEditModel
{
    public int? ProductId { get; set; }
    public string Article { get; set; }
    public string Name { get; set; }
    public int CategoryId { get; set; }
    public string Description { get; set; }
    public int ManufacturerId { get; set; }
    public string SupplierName { get; set; }
    public decimal Price { get; set; }
    public string UnitName { get; set; }
    public int StockQuantity { get; set; }
    public decimal DiscountPercent { get; set; }
    public string PhotoFile { get; set; }
}

public class OrderRow
{
    public int OrderId { get; set; }
    public int OrderNumber { get; set; }
    public string ArticlesText { get; set; }
    public string StatusName { get; set; }
    public string PickupAddress { get; set; }
    public DateTime? OrderDate { get; set; }
    public DateTime? DeliveryDate { get; set; }
}

public class OrderEditModel
{
    public int? OrderId { get; set; }
    public int OrderNumber { get; set; }
    public string ArticlesText { get; set; }
    public int StatusId { get; set; }
    public int PickupPointId { get; set; }
    public DateTime OrderDate { get; set; }
    public DateTime DeliveryDate { get; set; }
    public int PickupCode { get; set; }
}
}

```

Step 10 Create DataService.cs

Create единый слой доступа к БД для модулей 2-4.

Commands/code

Create файл DataService.cs:

- Обозреватель решений -> RMB по проекту -> Добавить -> Класс... -> имя DataService.cs.

```

using MySql.Data.MySqlClient;
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Windows.Media.Imaging;

namespace ShoeStore2026PUApp
{
    internal static class DataService
    {
        public static UserInfo Auth(string login, string password)
        {
            using (var conn = Db.GetConnection())
            {
                conn.Open();
            }
        }
    }
}

```

```

        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = @"
                SELECT u.user_id, u.full_name, r.role_name
                FROM users u
                JOIN roles r ON r.role_id = u.role_id
                WHERE u.login=@login AND u.password_plain=@password";
            cmd.Parameters.AddWithValue("@login", login);
            cmd.Parameters.AddWithValue("@password", password);

            using (var rd = cmd.ExecuteReader())
            {
                if (!rd.Read()) return null;
                return new UserInfo
                {
                    UserId = rd.GetInt32("user_id"),
                    FullName = rd.GetString("full_name"),
                    RoleName = rd.GetString("role_name")
                };
            }
        }
    }
}

public static List<ProductRow> GetProducts()
{
    var result = new List<ProductRow>();

    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = @"
                SELECT
                    p.product_id,
                    p.article,
                    p.name,
                    c.name AS category_name,
                    p.description_text,
                    m.name AS manufacturer_name,
                    s.name AS supplier_name,
                    p.price,
                    p.discount_percent,
                    p.unit_name,
                    p.stock_quantity,
                    p.photo_file
                FROM products p
                JOIN categories c ON c.category_id = p.category_id
                JOIN manufacturers m ON m.manufacturer_id = p.manufacturer_id
                JOIN suppliers s ON s.supplier_id = p.supplier_id
                ORDER BY p.product_id";

            using (var rd = cmd.ExecuteReader())
            {
                while (rd.Read())
                {
                    var price = ParseDecimal(rd["price"]);
                    var discount = ParseDecimal(rd["discount_percent"]);
                    var finalPrice = Math.Round(price * (100m - discount) / 100m, 2);

                    var photoFile = rd["photo_file"] == DBNull.Value ? "" : rd["photo_file"].ToString();
                    result.Add(new ProductRow
                    {
                        ProductId = Convert.ToInt32(rd["product_id"]),
                        Article = rd["article"].ToString(),
                        Name = rd["name"].ToString(),
                        Category = rd["category_name"].ToString(),
                        Description = rd["description_text"] == DBNull.Value ? "" :
                            rd["description_text"].ToString(),
                        Manufacturer = rd["manufacturer_name"].ToString(),
                        Supplier = rd["supplier_name"].ToString(),
                        Price = price,
                        DiscountPercent = discount,
                        FinalPrice = finalPrice,
                        UnitName = rd["unit_name"].ToString(),
                        StockQuantity = Convert.ToInt32(rd["stock_quantity"]),
                        HasDiscount = discount > 0m,
                        PhotoFile = photoFile,
                        PhotoImage = LoadPhoto(photoFile)
                    });
                }
            }
        }
    }
    return result;
}

public static List<string> GetSupplierNames()
{
    var result = new List<string>();
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {

```

```

        cmd.CommandText = "SELECT name FROM suppliers ORDER BY name";
        using (var rd = cmd.ExecuteReader())
        {
            while (rd.Read())
                result.Add(rd.GetString("name"));
        }
    }
    return result;
}

public static List<LookupItem> GetCategories()
{
    return GetLookup("SELECT category_id AS id, name FROM categories ORDER BY name");
}

public static List<LookupItem> GetManufacturers()
{
    return GetLookup("SELECT manufacturer_id AS id, name FROM manufacturers ORDER BY name");
}

public static List<LookupItem> GetStatuses()
{
    return GetLookup("SELECT status_id AS id, status_name AS name FROM order_statuses ORDER BY status_name");
}

public static List<LookupItem> GetPickupPoints()
{
    var result = new List<LookupItem>();
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = "SELECT pickup_point_id, address_text FROM pickup_points ORDER BY
            ↳ pickup_point_id";
            using (var rd = cmd.ExecuteReader())
            {
                while (rd.Read())
                {
                    result.Add(new LookupItem
                    {
                        Id = Convert.ToInt32(rd["pickup_point_id"]),
                        Name = rd["pickup_point_id"] + ". " + rd["address_text"]
                    });
                }
            }
        }
    }
    return result;
}

public static ProductEditModel GetProductById(int productId)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = @"
            SELECT
                p.product_id,
                p.article,
                p.name,
                p.category_id,
                p.description_text,
                p.manufacturer_id,
                s.name AS supplier_name,
                p.price,
                p.unit_name,
                p.stock_quantity,
                p.discount_percent,
                p.photo_file
            FROM products p
            JOIN suppliers s ON s.supplier_id = p.supplier_id
            WHERE p.product_id = @id";
            cmd.Parameters.AddWithValue("@id", productId);
            using (var rd = cmd.ExecuteReader())
            {
                if (!rd.Read()) return null;
                return new ProductEditModel
                {
                    ProductId = Convert.ToInt32(rd["product_id"]),
                    Article = rd["article"].ToString(),
                    Name = rd["name"].ToString(),
                    CategoryId = Convert.ToInt32(rd["category_id"]),
                    Description = rd["description_text"] == DBNull.Value ? "" :
                    ↳ rd["description_text"].ToString(),
                    ManufacturerId = Convert.ToInt32(rd["manufacturer_id"]),
                    SupplierName = rd["supplier_name"].ToString(),
                    Price = ParseDecimal(rd["price"]),
                    UnitName = rd["unit_name"].ToString(),
                    StockQuantity = Convert.ToInt32(rd["stock_quantity"]),
                    DiscountPercent = ParseDecimal(rd["discount_percent"]),
                    PhotoFile = rd["photo_file"] == DBNull.Value ? "" : rd["photo_file"].ToString()
                };
            }
        }
    }
}

```

```

    }
    }
}

public static void SaveProduct(ProductEditModel model)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var tx = conn.BeginTransaction())
        {
            try
            {
                var supplierId = EnsureSupplier(conn, tx, model.SupplierName);

                using (var cmd = conn.CreateCommand())
                {
                    cmd.Transaction = tx;
                    if (model.ProductId.HasValue)
                    {
                        cmd.CommandText = @"
UPDATE products
SET
    article=@article,
    name=@name,
    category_id=@categoryId,
    description_text=@description,
    manufacturer_id=@manufacturerId,
    supplier_id=@supplierId,
    price=@price,
    unit_name=@unitName,
    stock_quantity=@stock,
    discount_percent=@discount,
    photo_file=@photo
WHERE product_id=@id";
                        cmd.Parameters.AddWithValue("@id", model.ProductId.Value);
                    }
                    else
                    {
                        cmd.CommandText = @"
INSERT INTO products(
    article, name, category_id, description_text, manufacturer_id,
    supplier_id, price, unit_name, stock_quantity, discount_percent, photo_file
)
VALUES(
    @article, @name, @categoryId, @description, @manufacturerId,
    @supplierId, @price, @unitName, @stock, @discount, @photo
)";
                        cmd.Parameters.AddWithValue("@article", model.Article);
                        cmd.Parameters.AddWithValue("@name", model.Name);
                        cmd.Parameters.AddWithValue("@categoryId", model.CategoryId);
                        cmd.Parameters.AddWithValue("@description", string.IsNullOrEmpty(model.Description) ?
                            (object)DBNull.Value : model.Description);
                        cmd.Parameters.AddWithValue("@manufacturerId", model.ManufacturerId);
                        cmd.Parameters.AddWithValue("@supplierId", supplierId);
                        cmd.Parameters.AddWithValue("@price", model.Price);
                        cmd.Parameters.AddWithValue("@unitName", model.UnitName);
                        cmd.Parameters.AddWithValue("@stock", model.StockQuantity);
                        cmd.Parameters.AddWithValue("@discount", model.DiscountPercent);
                        cmd.Parameters.AddWithValue("@photo", string.IsNullOrEmpty(model.PhotoFile) ?
                            (object)DBNull.Value : model.PhotoFile);
                        cmd.ExecuteNonQuery();
                    }
                }
                tx.Commit();
            }
            catch
            {
                tx.Rollback();
                throw;
            }
        }
    }
}

public static bool ProductExistsInOrders(int productId)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = "SELECT COUNT(*) FROM order_items WHERE product_id=@id";
            cmd.Parameters.AddWithValue("@id", productId);
            return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
        }
    }
}

public static void DeleteProduct(int productId)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
    }
}

```

```

        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = "DELETE FROM products WHERE product_id=@id";
            cmd.Parameters.AddWithValue("@id", productId);
            cmd.ExecuteNonQuery();
        }
    }
}

public static List<OrderRow> GetOrders()
{
    var result = new List<OrderRow>();
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = @"
                SELECT
                    o.order_id,
                    o.order_number,
                    o.article_text,
                    os.status_name,
                    pp.address_text,
                    DATE_FORMAT(NULLIF(o.order_date, '0000-00-00'), '%Y-%m-%d') AS order_date_text,
                    DATE_FORMAT(NULLIF(o.delivery_date, '0000-00-00'), '%Y-%m-%d') AS delivery_date_text
                FROM orders o
                JOIN order_statuses os ON os.status_id = o.status_id
                JOIN pickup_points pp ON pp.pickup_point_id = o.pickup_point_id
                ORDER BY o.order_number";
            using (var rd = cmd.ExecuteReader())
            {
                while (rd.Read())
                {
                    result.Add(new OrderRow
                    {
                        OrderId = Convert.ToInt32(rd["order_id"]),
                        OrderNumber = Convert.ToInt32(rd["order_number"]),
                        ArticlesText = rd["article_text"].ToString(),
                        StatusName = rd["status_name"].ToString(),
                        PickupAddress = rd["address_text"].ToString(),
                        OrderDate = ParseNullableDate(rd["order_date_text"]),
                        DeliveryDate = ParseNullableDate(rd["delivery_date_text"])
                    });
                }
            }
        }
    }
    return result;
}

public static OrderEditModel GetOrderById(int orderId)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = @"
                SELECT
                    order_id, order_number, article_text, status_id,
                    pickup_point_id,
                    DATE_FORMAT(NULLIF(order_date, '0000-00-00'), '%Y-%m-%d') AS order_date_text,
                    DATE_FORMAT(NULLIF(delivery_date, '0000-00-00'), '%Y-%m-%d') AS delivery_date_text,
                    pickup_code
                FROM orders
                WHERE order_id = @id";
            cmd.Parameters.AddWithValue("@id", orderId);
            using (var rd = cmd.ExecuteReader())
            {
                if (!rd.Read()) return null;
                return new OrderEditModel
                {
                    OrderId = Convert.ToInt32(rd["order_id"]),
                    OrderNumber = Convert.ToInt32(rd["order_number"]),
                    ArticlesText = rd["article_text"].ToString(),
                    StatusId = Convert.ToInt32(rd["status_id"]),
                    PickupPointId = Convert.ToInt32(rd["pickup_point_id"]),
                    OrderDate = ParseNullableDate(rd["order_date_text"]) ?? DateTime.Today,
                    DeliveryDate = ParseNullableDate(rd["delivery_date_text"]) ?? DateTime.Today,
                    PickupCode = Convert.ToInt32(rd["pickup_code"])
                };
            }
        }
    }
}

public static int GetNextOrderNumber()
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = "SELECT COALESCE(MAX(order_number), 0) + 1 FROM orders";
            return Convert.ToInt32(cmd.ExecuteScalar());
        }
    }
}

```

```

    }
}

public static int GetNextPickupCode()
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = "SELECT COALESCE(MAX(pickup_code), 900) + 1 FROM orders";
            return Convert.ToInt32(cmd.ExecuteScalar());
        }
    }
}

public static void SaveOrder(OrderEditModel model)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var tx = conn.BeginTransaction())
        {
            try
            {
                var pairs = ParseOrderArticles(model.ArticlesText);
                var articleMap = ResolveArticleMap(conn, tx, pairs);

                int orderId = model.OrderId ?? 0;
                using (var cmd = conn.CreateCommand())
                {
                    cmd.Transaction = tx;
                    if (model.OrderId.HasValue)
                    {
                        cmd.CommandText = @"
UPDATE orders
SET
    article_text=@articles,
    status_id=@statusId,
    pickup_point_id=@pickupId,
    order_date=@orderDate,
    delivery_date=@deliveryDate
WHERE order_id=@id";
                        cmd.Parameters.AddWithValue("@id", model.OrderId.Value);
                    }
                    else
                    {
                        cmd.CommandText = @"
INSERT INTO orders(
    order_number, article_text, order_date, delivery_date,
    pickup_point_id, client_user_id, pickup_code, status_id
)
VALUES(
    @orderNumber, @articles, @orderDate, @deliveryDate,
    @pickupId, NULL, @pickupCode, @statusId
)";
                        cmd.Parameters.AddWithValue("@orderNumber", model.OrderNumber);
                        cmd.Parameters.AddWithValue("@pickupCode", model.PickupCode);
                    }

                    cmd.Parameters.AddWithValue("@articles", model.ArticlesText);
                    cmd.Parameters.AddWithValue("@statusId", model.StatusId);
                    cmd.Parameters.AddWithValue("@pickupId", model.PickupPointId);
                    cmd.Parameters.AddWithValue("@orderDate", model.OrderDate.Date);
                    cmd.Parameters.AddWithValue("@deliveryDate", model.DeliveryDate.Date);
                    cmd.ExecuteNonQuery();

                    if (!model.OrderId.HasValue)
                        orderId = (int)cmd.LastInsertedId;
                }

                using (var cmd = conn.CreateCommand())
                {
                    cmd.Transaction = tx;
                    cmd.CommandText = "DELETE FROM order_items WHERE order_id=@orderId";
                    cmd.Parameters.AddWithValue("@orderId", orderId);
                    cmd.ExecuteNonQuery();
                }

                foreach (var p in pairs)
                {
                    using (var cmd = conn.CreateCommand())
                    {
                        cmd.Transaction = tx;
                        cmd.CommandText = "INSERT INTO order_items(order_id, product_id, quantity)
VALUES(@orderId,@productId,@qty)";
                        cmd.Parameters.AddWithValue("@orderId", orderId);
                        cmd.Parameters.AddWithValue("@productId", articleMap[p.Article]);
                        cmd.Parameters.AddWithValue("@qty", p.Quantity);
                        cmd.ExecuteNonQuery();
                    }
                }

                tx.Commit();
            }
        }
    }
}

```

```

        catch
        {
            tx.Rollback();
            throw;
        }
    }
}

public static void DeleteOrder(int orderId)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        using (var cmd = conn.CreateCommand())
        {
            cmd.CommandText = "DELETE FROM orders WHERE order_id=@id";
            cmd.Parameters.AddWithValue("@id", orderId);
            cmd.ExecuteNonQuery();
        }
    }
}

private static List<OrderArticlePair> ParseOrderArticles(string text)
{
    var tokens = (text ?? "")
        .Split(new[] { ',' }, StringSplitOptions.RemoveEmptyEntries)
        .Select(x => x.Trim())
        .Where(x => x.Length > 0)
        .ToList();

    if (tokens.Count < 2 || tokens.Count % 2 != 0)
        throw new Exception("Поле артикулов задается парами: артикул, количество.");

    var result = new List<OrderArticlePair>();
    for (int i = 0; i < tokens.Count; i += 2)
    {
        int qty;
        if (!int.TryParse(tokens[i + 1], out qty))
            throw new Exception("Количество в паре артикулов должно быть целым числом.");
        if (qty <= 0)
            throw new Exception("Количество в паре артикулов должно быть больше 0.");

        result.Add(new OrderArticlePair
        {
            Article = tokens[i],
            Quantity = qty
        });
    }
    return result;
}

private static Dictionary<string, int> ResolveArticleMap(MySqlConnection conn, MySqlTransaction tx,
    List<OrderArticlePair> pairs)
{
    var uniqueArticles = pairs.Select(x => x.Article).Distinct().ToList();
    var result = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase);

    using (var cmd = conn.CreateCommand())
    {
        {
            cmd.Transaction = tx;
            var paramNames = new List<string>();
            for (int i = 0; i < uniqueArticles.Count; i++)
            {
                var p = "@a" + i;
                paramNames.Add(p);
                cmd.Parameters.AddWithValue(p, uniqueArticles[i]);
            }
            cmd.CommandText = "SELECT article, product_id FROM products WHERE article IN (" + string.Join(",",
                paramNames) + ")";
            using (var rd = cmd.ExecuteReader())
            {
                while (rd.Read())
                    result[rd["article"].ToString()] = Convert.ToInt32(rd["product_id"]);
            }
        }

        var missing = uniqueArticles.Where(x => !result.ContainsKey(x)).ToList();
        if (missing.Count > 0)
            throw new Exception("В таблице products не найдены артикулы: " + string.Join(", ", missing));

        return result;
    }
}

private static int EnsureSupplier(MySqlConnection conn, MySqlTransaction tx, string supplierName)
{
    using (var cmd = conn.CreateCommand())
    {
        {
            cmd.Transaction = tx;
            cmd.CommandText = "SELECT supplier_id FROM suppliers WHERE name=@name";
            cmd.Parameters.AddWithValue("@name", supplierName);
            var exists = cmd.ExecuteScalar();
            if (exists != null && exists != DBNull.Value)
                return Convert.ToInt32(exists);
        }
    }
}

```

```

        using (var cmd = conn.CreateCommand())
        {
            cmd.Transaction = tx;
            cmd.CommandText = "INSERT INTO suppliers(name) VALUES(@name)";
            cmd.Parameters.AddWithValue("@name", supplierName);
            cmd.ExecuteNonQuery();
            return (int)cmd.LastInsertedId;
        }
    }

    private static List<LookupItem> GetLookup(string sql)
    {
        var result = new List<LookupItem>();
        using (var conn = Db.GetConnection())
        {
            conn.Open();
            using (var cmd = conn.CreateCommand())
            {
                cmd.CommandText = sql;
                using (var rd = cmd.ExecuteReader())
                {
                    while (rd.Read())
                    {
                        result.Add(new LookupItem
                        {
                            Id = Convert.ToInt32(rd["id"]),
                            Name = rd["name"].ToString()
                        });
                    }
                }
            }
        }
        return result;
    }

    private static decimal ParseDecimal(object value)
    {
        if (value == null || value == DBNull.Value) return 0m;
        return Convert.ToDecimal(value, CultureInfo.InvariantCulture);
    }

    private static DateTime? ParseNullableDate(object value)
    {
        if (value == null || value == DBNull.Value) return null;
        var text = value.ToString();
        if (string.IsNullOrEmpty(text)) return null;

        DateTime date;
        if (DateTime.TryParse(text, out date))
            return date.Date;

        return null;
    }

    private static BitmapImage LoadPhoto(string fileName)
    {
        if (!string.IsNullOrEmpty(fileName))
        {
            var cleanName = Path.GetFileName(fileName.Trim());
            var runtimePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "resources", "photos",
                cleanName);
            if (File.Exists(runtimePath))
            {
                return LoadBitmap(new Uri(runtimePath, UriKind.Absolute));
            }

            var packPhoto = $"pack://application:,,,/resources/photos/{cleanName}";
            try
            {
                return LoadBitmap(new Uri(packPhoto, UriKind.Absolute));
            }
            catch
            {
            }
        }
        return LoadBitmap(new Uri("pack://application:,,,/resources/picture.png", UriKind.Absolute));
    }

    private static BitmapImage LoadBitmap(Uri uri)
    {
        var image = new BitmapImage();
        image.BeginInit();
        image.CacheOption = BitmapCacheOption.OnLoad;
        image.UriSource = uri;
        image.EndInit();
        image.Freeze();
        return image;
    }

    private class OrderArticlePair
    {
        public string Article { get; set; }
        public int Quantity { get; set; }
    }
}

```


Step 11 Create LoginWindow и setting старт приложения

Create окно входа (логин/пароль + гость) и set его стартовым.

Command/code

Обозреватель решений -> RMB по проекту -> Добавить -> Окно (WPF)... -> имя LoginWindow.xaml.

Replace LoginWindow.xaml:

```
<Window x:Class="ShoeStore2026PUApp.LoginWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Вход"
        Height="320"
        Width="520"
        FontFamily="Times New Roman"
        Background="#FFFFFF"
        Icon="resources/Icon.ico"
        WindowStartupLocation="CenterScreen">
    <Grid Margin="18">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock Text="Логин" FontSize="16" Margin="0,0,0,6"/>
        <TextBox x:Name="LoginTextBox" Grid.Row="1" Height="34" FontSize="16" Margin="0,0,0,12"/>

        <TextBlock Text="Пароль" Grid.Row="2" FontSize="16" Margin="0,0,0,6"/>
        <PasswordBox x:Name="PasswordTextBox" Grid.Row="3" Height="34" FontSize="16" Margin="0,0,0,12"/>

        <StackPanel Grid.Row="4" Orientation="Horizontal" HorizontalAlignment="Right">
            <Button Content="Войти" Width="130" Height="36" Margin="0,0,10,0" Background="#00FA9A"
                Click="Login_Click"/>
            <Button Content="Войти как гость" Width="150" Height="36" Background="#00FA9A" Click="Guest_Click"/>
        </StackPanel>
    </Grid>
</Window>
```

Replace LoginWindow.xaml.cs:

```
using System;
using System.Windows;

namespace ShoeStore2026PUApp
{
    public partial class LoginWindow : Window
    {
        public LoginWindow()
        {
            InitializeComponent();
        }

        private void Login_Click(object sender, RoutedEventArgs e)
        {
            var login = LoginTextBox.Text.Trim();
            var password = PasswordTextBox.Password.Trim();
            if (string.IsNullOrEmpty(login) || string.IsNullOrEmpty(password))
            {
                MessageBox.Show("Введите логин и пароль.", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            try
            {
                var user = DataService.Auth(login, password);
                if (user == null)
                {
                    MessageBox.Show("Неверный логин или пароль.", "Ошибка входа", MessageBoxButton.OK,
                        MessageBoxImage.Error);
                    return;
                }
                OpenMain(user);
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Ошибка подключения к БД", MessageBoxButton.OK, MessageBoxImage.Error);
            }
        }

        private void Guest_Click(object sender, RoutedEventArgs e)
        {
            // ...
        }
    }
}
```

```

        var guest = new UserInfo
        {
            UserId = null,
            FullName = "Гость",
            RoleName = "Гость"
        };
        OpenMain(guest);
    }

    private void OpenMain(UserInfo user)
    {
        var wnd = new MainWindow(user);
        wnd.Show();
        Close();
    }
}
}

```

Replace App.xaml:

```

<Application x:Class="ShoeStore2026PUApp.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    StartupUri="LoginWindow.xaml">
    <Application.Resources>
    </Application.Resources>
</Application>

```

Step 12 Create окно ProductFormWindow

Create форму добавления/редактирования товара с выбором фото до 300x200.

Command/code

Обозреватель решений -> RMB по проекту -> Добавить -> Окно (WPF)... -> имя ProductFormWindow.xaml.

Replace ProductFormWindow.xaml:

```

<Window x:Class="ShoeStore2026PUApp.ProductFormWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Добавление/редактирование товара"
    Height="780"
    Width="980"
    FontFamily="Times New Roman"
    Background="#FFFFFF"
    Icon="resources/Icon.ico"
    WindowStartupLocation="CenterOwner">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>

        <Grid Grid.Row="0">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="2*" />
                <ColumnDefinition Width="1*" />
            </Grid.ColumnDefinitions>

            <Grid Grid.Column="0" Margin="0,0,14,0">
                <Grid.RowDefinitions>
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="220" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>

                <TextBlock x:Name="IdLabel" Text="ID товара:" VerticalAlignment="Center" />
                <TextBox x:Name="IdTextBox" Grid.Column="1" IsReadOnly="True" Margin="0,0,0,8" />

                <TextBlock Grid.Row="1" Text="Артикул:" VerticalAlignment="Center" />
                <TextBox x:Name="ArticleTextBox" Grid.Row="1" Grid.Column="1" Margin="0,0,0,8" />

                <TextBlock Grid.Row="2" Text="Наименование товара:" VerticalAlignment="Center" />
            </Grid>
        </Grid>
    </Grid>

```

```

        <TextBox x:Name="NameTextBox" Grid.Row="2" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="3" Text="Категория товара:" VerticalAlignment="Center"/>
        <ComboBox x:Name="CategoryComboBox" Grid.Row="3" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="4" Text="Описание товара:" VerticalAlignment="Top"/>
        <TextBox x:Name="DescriptionTextBox" Grid.Row="4" Grid.Column="1" Height="90" TextWrapping="Wrap"
        ↳ AcceptsReturn="True" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="5" Text="Производитель:" VerticalAlignment="Center"/>
        <ComboBox x:Name="ManufacturerComboBox" Grid.Row="5" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="6" Text="Поставщик:" VerticalAlignment="Center"/>
        <TextBox x:Name="SupplierTextBox" Grid.Row="6" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="7" Text="Цена:" VerticalAlignment="Center"/>
        <TextBox x:Name="PriceTextBox" Grid.Row="7" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="8" Text="Единица измерения:" VerticalAlignment="Center"/>
        <TextBox x:Name="UnitTextBox" Grid.Row="8" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="9" Text="Кол-во на складе:" VerticalAlignment="Center"/>
        <TextBox x:Name="StockTextBox" Grid.Row="9" Grid.Column="1" Margin="0,0,0,8"/>

        <TextBlock Grid.Row="10" Text="Действующая скидка (%):" VerticalAlignment="Center"/>
        <TextBox x:Name="DiscountTextBox" Grid.Row="10" Grid.Column="1" Margin="0,0,0,8"/>
    </Grid>

    <StackPanel Grid.Column="1">
        <TextBlock Text="Фото" Margin="0,0,0,6"/>
        <Border BorderBrush="#2E8B57" BorderThickness="1" Width="300" Height="200">
            <Image x:Name="PhotoImage" Stretch="Uniform"/>
        </Border>
        <Button Content="Выбрать фото" Width="170" Height="34" Margin="0,10,0,0" Background="#7FFF00"
        ↳ Click="ChoosePhoto_Click"/>
    </StackPanel>
</Grid>

<StackPanel Grid.Row="1" Orientation="Horizontal" HorizontalAlignment="Right" Margin="0,12,0,0">
    <Button Content="Сохранить" Width="150" Height="36" Margin="0,0,10,0" Background="#00FA9A"
    ↳ Click="Save_Click"/>
    <Button Content="Назад" Width="150" Height="36" Background="#7FFF00" Click="Back_Click"/>
</StackPanel>
</Grid>
</Window>

```

Replace ProductFormWindow.xaml.cs:

```

using Microsoft.Win32;
using System;
using System.Globalization;
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;

namespace ShoeStore2026PUApp
{
    public partial class ProductFormWindow : Window
    {
        private readonly int? _productId;
        private string _oldPhotoFile = "";
        private string _selectedPhotoPath = "";

        public ProductFormWindow(int? productId = null)
        {
            InitializeComponent();
            _productId = productId;
            LoadLookups();

            if (_productId.HasValue)
            {
                LoadProduct(_productId.Value);
            }
            else
            {
                IdLabel.Visibility = Visibility.Collapsed;
                IdTextBox.Visibility = Visibility.Collapsed;
                SetPreviewFromPack("pack://application:,,,/resources/picture.png");
            }
        }

        private void LoadLookups()
        {
            CategoryComboBox.DisplayMemberPath = "Name";
            CategoryComboBox.SelectedValuePath = "Id";
            CategoryComboBox.ItemsSource = DataService.GetCategories();

            ManufacturerComboBox.DisplayMemberPath = "Name";
            ManufacturerComboBox.SelectedValuePath = "Id";
            ManufacturerComboBox.ItemsSource = DataService.GetManufacturers();
        }

        private void LoadProduct(int productId)
        {
            var row = DataService.GetProductById(productId);

```

```

        if (row == null)
        {
            MessageBox.Show("Товар не найден.", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
            Close();
            return;
        }

        IdTextBox.Text = row.ProductId.Value.ToString();
        ArticleTextBox.Text = row.Article;
        NameTextBox.Text = row.Name;
        DescriptionTextBox.Text = row.Description;
        SupplierTextBox.Text = row.SupplierName;
        PriceTextBox.Text = row.Price.ToString("0.##", CultureInfo.InvariantCulture);
        UnitTextBox.Text = row.UnitName;
        StockTextBox.Text = row.StockQuantity.ToString();
        DiscountTextBox.Text = row.DiscountPercent.ToString("0.##", CultureInfo.InvariantCulture);

        CategoryComboBox.SelectedValue = row.CategoryId;
        ManufacturerComboBox.SelectedValue = row.ManufacturerId;

        _oldPhotoFile = row.PhotoFile ?? "";
        if (!string.IsNullOrEmpty(_oldPhotoFile))
        {
            var f = Path.GetFileName(_oldPhotoFile);
            var runtimePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "resources", "photos", f);
            if (File.Exists(runtimePath))
            {
                var runtimeImage = LoadBitmap(runtimePath);
                PhotoImage.Source = runtimeImage;
            }
            else
            {
                SetPreviewFromPack($"pack://application:,,,/resources/photos/{f}");
            }
        }
        else
        {
            SetPreviewFromPack("pack://application:,,,/resources/picture.png");
        }
    }

    private void ChoosePhoto_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog
        {
            Filter = "Image files|*.png;*.jpg;*.jpeg;*.bmp"
        };
        if (dialog.ShowDialog() != true) return;

        var bitmap = LoadBitmap(dialog.FileName);
        if (bitmap == null)
        {
            MessageBox.Show("Не удалось прочитать изображение.", "Ошибка", MessageBoxButton.OK,
                MessageBoxImage.Warning);
            return;
        }

        if (bitmap.PixelWidth > 300 || bitmap.PixelHeight > 200)
        {
            MessageBox.Show("Размер фото не должен превышать 300x200 пикселей.", "Ограничение",
                MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        _selectedPhotoPath = dialog.FileName;
        PhotoImage.Source = bitmap;
    }

    private void Save_Click(object sender, RoutedEventArgs e)
    {
        var article = ArticleTextBox.Text.Trim();
        var name = NameTextBox.Text.Trim();
        var supplier = SupplierTextBox.Text.Trim();
        var unit = UnitTextBox.Text.Trim();
        var description = DescriptionTextBox.Text.Trim();

        if (string.IsNullOrEmpty(article) ||
            string.IsNullOrEmpty(name) ||
            string.IsNullOrEmpty(supplier) ||
            string.IsNullOrEmpty(unit))
        {
            MessageBox.Show("Заполните обязательные поля: артикул, наименование, поставщик, единица измерения.",
                "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        if (!(CategoryComboBox.SelectedItem is LookupItem category) ||
            !(ManufacturerComboBox.SelectedItem is LookupItem manufacturer))
        {
            MessageBox.Show("Выберите категорию и производителя.", "Ошибка", MessageBoxButton.OK,
                MessageBoxImage.Warning);
            return;
        }

        if (!TryParseDecimal(PriceTextBox.Text, out var price) ||
            !TryParseDecimal(DiscountTextBox.Text, out var discount))
    }

```

```

    {
        MessageBox.Show("Проверьте формат цены и скидки.", "Ошибка", MessageBoxButton.OK,
            MessageBoxIcon.Warning);
        return;
    }

    if (!int.TryParse(StockTextBox.Text.Trim(), out var stock))
    {
        MessageBox.Show("Количество на складе должно быть целым числом.", "Ошибка", MessageBoxButton.OK,
            MessageBoxIcon.Warning);
        return;
    }

    if (price < 0 || discount < 0 || stock < 0)
    {
        MessageBox.Show("Цена, скидка и количество не могут быть отрицательными.",
            "Ошибка", MessageBoxButton.OK, MessageBoxIcon.Warning);
        return;
    }

    var photoFile = _oldPhotoFile;
    if (!string.IsNullOrEmpty(_selectedPhotoPath))
    {
        var copied = CopyPhotoToProject(_selectedPhotoPath);
        if (string.IsNullOrEmpty(copied))
        {
            MessageBox.Show("Не удалось сохранить изображение.", "Ошибка", MessageBoxButton.OK,
                MessageBoxIcon.Error);
            return;
        }

        if (!string.IsNullOrEmpty(_oldPhotoFile))
        {
            DeleteOldPhoto(_oldPhotoFile);
        }
        photoFile = copied;
    }

    var model = new ProductEditModel
    {
        ProductId = _productId,
        Article = article,
        Name = name,
        CategoryId = category.Id,
        Description = description,
        ManufacturerId = manufacturer.Id,
        SupplierName = supplier,
        Price = price,
        UnitName = unit,
        StockQuantity = stock,
        DiscountPercent = discount,
        PhotoFile = photoFile
    };

    try
    {
        DataService.SaveProduct(model);
        DialogResult = true;
        Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Ошибка сохранения", MessageBoxButton.OK, MessageBoxIcon.Error);
    }
}

private void Back_Click(object sender, RoutedEventArgs e)
{
    DialogResult = false;
    Close();
}

private bool TryParseDecimal(string text, out decimal value)
{
    text = (text ?? "").Trim().Replace(',', '.');
    return decimal.TryParse(text, NumberStyles.Any, CultureInfo.InvariantCulture, out value);
}

private string CopyPhotoToProject(string sourcePath)
{
    try
    {
        var ext = Path.GetExtension(sourcePath);
        if (string.IsNullOrEmpty(ext)) ext = ".png";
        var fileName = "uploaded_" + DateTime.Now.Ticks + ext.ToLowerInvariant();

        var baseDir = AppDomain.CurrentDomain.BaseDirectory;
        var targetDir = Path.Combine(baseDir, "resources", "photos");
        if (!Directory.Exists(targetDir))
            Directory.CreateDirectory(targetDir);

        var targetPath = Path.Combine(targetDir, fileName);
        File.Copy(sourcePath, targetPath, true);
        return "resources/photos/" + fileName;
    }
    catch

```

```

        {
            return "";
        }
    }

    private void DeleteOldPhoto(string oldPhotoFile)
    {
        try
        {
            var name = Path.GetFileName(oldPhotoFile);
            if (string.IsNullOrEmpty(name)) return;

            var baseDir = AppDomain.CurrentDomain.BaseDirectory;
            var path = Path.Combine(baseDir, "resources", "photos", name);
            if (File.Exists(path))
                File.Delete(path);
        }
        catch
        {
        }
    }

    private void SetPreviewFromPack(string packUri)
    {
        try
        {
            PhotoImage.Source = new BitmapImage(new Uri(packUri, UriKind.Absolute));
        }
        catch
        {
            PhotoImage.Source = null;
        }
    }

    private BitmapImage LoadBitmap(string filePath)
    {
        try
        {
            var image = new BitmapImage();
            image.BeginInit();
            image.CacheOption = BitmapCacheOption.OnLoad;
            image.UriSource = new Uri(filePath, UriKind.Absolute);
            image.EndInit();
            image.Freeze();
            return image;
        }
        catch
        {
            return null;
        }
    }
}

```

Step 13 Create окно OrdersWindow

Create окно списка заказов для менеджера и администратора.

Command/code

Обозреватель решений -> RMB по проекту -> Добавить -> Окно (WPF)... -> имя OrdersWindow.xaml.

Replace OrdersWindow.xaml:

```

<Window x:Class="ShoeStore2026PUApp.OrdersWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Заказы"
        Height="700"
        Width="1200"
        FontFamily="Times New Roman"
        Background="#FFFFFF"
        Icon="resources/Icon.ico"
        WindowStartupLocation="CenterOwner">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <StackPanel Orientation="Horizontal" Margin="0,0,0,8">
            <TextBlock Text="Список заказов" FontSize="24" FontWeight="Bold" VerticalAlignment="Center"/>
            <StackPanel Orientation="Horizontal" HorizontalAlignment="Right" Margin="20,0,0,0">
                <Button x:Name="AddOrderButton" Content="Добавить заказ" Width="160" Height="34" Margin="0,0,8,0"
                    &#x2D; Background="#00FA9A" Click="AddOrder_Click"/>
                <Button x:Name="EditOrderButton" Content="Редактировать заказ" Width="180" Height="34"
                    &#x2D; Margin="0,0,8,0" Background="#7FFF00" Click="EditOrder_Click"/>
                <Button x:Name="DeleteOrderButton" Content="Удалить заказ" Width="160" Height="34" Margin="0,0,8,0"

```

```

        <Button Content="Назад" Width="120" Height="34" Background="#7FFF00" Click="Back_Click"/>
    </StackPanel>
</StackPanel>

<DataGrid Grid.Row="1"
    x:Name="OrdersGrid"
    AutoGenerateColumns="False"
    IsReadOnly="True"
    HeadersVisibility="Column"
    MouseDoubleClick="OrdersGrid_MouseDoubleClick">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Номер" Binding="{Binding OrderNumber}" Width="110"/>
        <DataGridTextColumn Header="Артикул заказа" Binding="{Binding ArticlesText}" Width="260"/>
        <DataGridTextColumn Header="Статус" Binding="{Binding StatusName}" Width="170"/>
        <DataGridTextColumn Header="Пункт выдачи" Binding="{Binding PickupAddress}" Width="290"/>
        <DataGridTextColumn Header="Дата заказа" Binding="{Binding OrderDate, StringFormat={}{0:yyyy-MM-dd}}"
            < Width="140"/>
        <DataGridTextColumn Header="Дата выдачи" Binding="{Binding DeliveryDate,
            < StringFormat={}{0:yyyy-MM-dd}}" Width="140"/>
    </DataGrid.Columns>
</DataGrid>
</Grid>
</Window>

```

Replace OrdersWindow.xaml.cs:

```

using System;
using System.Linq;
using System.Windows;

namespace ShoeStore2026PUApp
{
    public partial class OrdersWindow : Window
    {
        private readonly UserInfo _currentUser;

        public OrdersWindow(UserInfo currentUser)
        {
            InitializeComponent();
            _currentUser = currentUser;

            var isAdmin = string.Equals(_currentUser.RoleName, "Администратор", StringComparison.OrdinalIgnoreCase);
            AddOrderButton.Visibility = isAdmin ? Visibility.Visible : Visibility.Collapsed;
            EditOrderButton.Visibility = isAdmin ? Visibility.Visible : Visibility.Collapsed;
            DeleteOrderButton.Visibility = isAdmin ? Visibility.Visible : Visibility.Collapsed;

            LoadOrders();
        }

        private void LoadOrders()
        {
            try
            {
                OrdersGrid.ItemsSource = DataService.GetOrders();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Ошибка БД", MessageBoxButton.OK, MessageBoxImage.Error);
            }
        }

        private bool IsAdmin()
        {
            return string.Equals(_currentUser.RoleName, "Администратор", StringComparison.OrdinalIgnoreCase);
        }

        private void AddOrder_Click(object sender, RoutedEventArgs e)
        {
            if (!IsAdmin()) return;
            var wnd = new OrderFormWindow(null) { Owner = this };
            if (wnd.ShowDialog() == true)
                LoadOrders();
        }

        private void EditOrder_Click(object sender, RoutedEventArgs e)
        {
            if (!IsAdmin()) return;

            var row = OrdersGrid.SelectedItem as OrderRow;
            if (row == null)
            {
                MessageBox.Show("Выберите заказ для редактирования.", "Информация", MessageBoxButton.OK,
                    < MessageBoxImage.Information);
                return;
            }

            var wnd = new OrderFormWindow(row.OrderId) { Owner = this };
            if (wnd.ShowDialog() == true)
                LoadOrders();
        }

        private void OrdersGrid_MouseDoubleClick(object sender, System.Windows.Input.MouseButtonEventArgs e)
        {
            if (IsAdmin())

```

```

        EditOrder_Click(sender, e);
    }

    private void DeleteOrder_Click(object sender, RoutedEventArgs e)
    {
        if (!IsAdmin()) return;

        var row = OrdersGrid.SelectedItem as OrderRow;
        if (row == null)
        {
            MessageBox.Show("Выберите заказ для удаления.", "Информация", MessageBoxButton.OK,
                MessageBoxImage.Information);
            return;
        }

        var confirm = MessageBox.Show("Удалить выбранный заказ?", "Подтверждение",
            MessageBoxButton.YesNo, MessageBoxImage.Question);
        if (confirm != MessageBoxResult.Yes) return;

        try
        {
            DataService.DeleteOrder(row.OrderId);
            LoadOrders();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Ошибка удаления", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    private void Back_Click(object sender, RoutedEventArgs e)
    {
        Close();
    }
}
}

```

Step 14 Create окно OrderFormWindow

Create форму добавления/редактирования заказа.

Command/code

Обозреватель решений -> RMB по проекту -> Добавить -> Окно (WPF)... -> имя OrderFormWindow.xaml.

Replace OrderFormWindow.xaml:

```

<Window x:Class="ShoeStore2026PUApp.OrderFormWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Добавление/редактирование заказа"
    Height="460"
    Width="760"
    FontFamily="Times New Roman"
    Background="#FFFFFF"
    Icon="resources/Icon.ico"
    WindowStartupLocation="CenterOwner">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>

        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="220" />
                <ColumnDefinition Width="*" />
            </Grid.ColumnDefinitions>

            <TextBlock Grid.Row="0" Grid.Column="0" Text="Номер заказа:" VerticalAlignment="Center" Margin="0,0,0,8"/>
            <TextBox x:Name="OrderNumberTextBox" Grid.Row="0" Grid.Column="1" Margin="0,0,0,8" IsReadOnly="True"/>

            <TextBlock Grid.Row="1" Grid.Column="0" Text="Артикул:" VerticalAlignment="Center" Margin="0,0,0,8"/>
            <TextBox x:Name="ArticlesTextBox" Grid.Row="1" Grid.Column="1" Margin="0,0,0,8"/>

            <TextBlock Grid.Row="2" Grid.Column="0" Text="Статус заказа:" VerticalAlignment="Center"
                Margin="0,0,0,8"/>
            <ComboBox x:Name="StatusComboBox" Grid.Row="2" Grid.Column="1" Margin="0,0,0,8"/>

            <TextBlock Grid.Row="3" Grid.Column="0" Text="Адрес пункта выдачи:" VerticalAlignment="Center"

```



```

        Margin="0,0,0,8"/>
<ComboBox x:Name="PickupComboBox" Grid.Row="3" Grid.Column="1" Margin="0,0,0,8"/>

<TextBlock Grid.Row="4" Grid.Column="0" Text="Дата заказа:" VerticalAlignment="Center" Margin="0,0,0,8"/>
<DatePicker x:Name="OrderDatePicker" Grid.Row="4" Grid.Column="1" Margin="0,0,0,8"/>

<TextBlock Grid.Row="5" Grid.Column="0" Text="Дата выдачи:" VerticalAlignment="Center" Margin="0,0,0,8"/>
<DatePicker x:Name="DeliveryDatePicker" Grid.Row="5" Grid.Column="1" Margin="0,0,0,8"/>
</Grid>

<StackPanel Grid.Row="1" Orientation="Horizontal" HorizontalAlignment="Right" Margin="0,12,0,0">
    <Button Content="Сохранить" Width="150" Height="36" Margin="0,0,10,0" Background="#00FA9A"
        Click="Save_Click"/>
    <Button Content="Назад" Width="150" Height="36" Background="#7FFF00" Click="Back_Click"/>
</StackPanel>
</Grid>
</Window>

```

Replace OrderFormWindow.xaml.cs:

```

using System;
using System.Windows;

namespace ShoeStore2026PUApp
{
    public partial class OrderFormWindow : Window
    {
        private readonly int? _orderId;
        private int _pickupCode;

        public OrderFormWindow(int? orderId = null)
        {
            InitializeComponent();
            _orderId = orderId;

            LoadLookups();
            if (_orderId.HasValue)
            {
                LoadOrder(_orderId.Value);
            }
            else
            {
                OrderNumberTextBox.Text = DataService.GetNextOrderNumber().ToString();
                _pickupCode = DataService.GetNextPickupCode();
                OrderDatePicker.SelectedDate = DateTime.Today;
                DeliveryDatePicker.SelectedDate = DateTime.Today.AddDays(1);
            }
        }

        private void LoadLookups()
        {
            StatusComboBox.DisplayMemberPath = "Name";
            StatusComboBox.SelectedValuePath = "Id";
            StatusComboBox.ItemsSource = DataService.GetStatuses();

            PickupComboBox.DisplayMemberPath = "Name";
            PickupComboBox.SelectedValuePath = "Id";
            PickupComboBox.ItemsSource = DataService.GetPickupPoints();
        }

        private void LoadOrder(int orderId)
        {
            var row = DataService.GetOrderById(orderId);
            if (row == null)
            {
                MessageBox.Show("Заказ не найден.", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
                Close();
                return;
            }

            OrderNumberTextBox.Text = row.OrderNumber.ToString();
            ArticlesTextBox.Text = row.ArticlesText;
            StatusComboBox.SelectedValue = row.StatusId;
            PickupComboBox.SelectedValue = row.PickupPointId;
            OrderDatePicker.SelectedDate = row.OrderDate;
            DeliveryDatePicker.SelectedDate = row.DeliveryDate;
            _pickupCode = row.PickupCode;
        }

        private void Save_Click(object sender, RoutedEventArgs e)
        {
            var articles = ArticlesTextBox.Text.Trim();
            if (string.IsNullOrEmpty(articles))
            {
                MessageBox.Show("Поле Артикул обязательно.", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            if (!(StatusComboBox.SelectedItem is LookupItem status) ||
                !(PickupComboBox.SelectedItem is LookupItem pickup))
            {
                MessageBox.Show("Выберите статус и пункт выдачи.", "Ошибка", MessageBoxButton.OK,
                    Click = "Save_Click"/>
                    <Button Content="Назад" Width="150" Height="36" Background="#7FFF00" Click="Back_Click"/>
                </StackPanel>
                </Grid>
            </Window>
        }
    }
}

```

```

        if (!OrderDatePicker.SelectedDate.HasValue || !DeliveryDatePicker.SelectedDate.HasValue)
        {
            MessageBox.Show("Укажите дату заказа и дату выдачи.", "Ошибка", MessageBoxButton.OK,
                MessageBoxImage.Warning);
            return;
        }

        var model = new OrderEditModel
        {
            OrderId = _orderId,
            OrderNumber = int.Parse(OrderNumberTextBox.Text),
            ArticlesText = articles,
            StatusId = status.Id,
            PickupPointId = pickup.Id,
            OrderDate = OrderDatePicker.SelectedDate.Value.Date,
            DeliveryDate = DeliveryDatePicker.SelectedDate.Value.Date,
            PickupCode = _pickupCode
        };

        try
        {
            DataService.SaveOrder(model);
            DialogResult = true;
            Close();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Ошибка сохранения", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    private void Back_Click(object sender, RoutedEventArgs e)
    {
        DialogResult = false;
        Close();
    }
}
}

```

Step 15 Create MainWindow

Create главное окно со списком товаров, ролями и переходом к заказам.

Command/code

Replace MainWindow.xaml:

```

<Window x:Class="ShoeStore2026PUApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Список товаров"
        Height="820"
        Width="1480"
        FontFamily="Times New Roman"
        Background="#FFFFFF"
        Icon="resources/Icon.ico"
        WindowStartupLocation="CenterScreen">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/"/>
        </Grid.RowDefinitions>

        <Grid Grid.Row="0" Margin="0,0,0,10">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="Auto"/>
                <ColumnDefinition Width="*/"/>
                <ColumnDefinition Width="Auto"/>
            </Grid.ColumnDefinitions>

            <Image Source="resources/Icon.png" Width="120" Height="70" Stretch="Uniform" Margin="0,0,10,0"/>

            <TextBlock Grid.Column="1"
                Text="000 «Обувь» – Список товаров"
                VerticalAlignment="Center"
                FontSize="28"
                FontWeight="Bold"/>

            <StackPanel Grid.Column="2" HorizontalAlignment="Right">
                <TextBlock x:Name="RoleTextBlock" FontSize="16" TextAlignment="Right"/>
                <TextBlock x:Name="UserTextBlock" FontSize="16" TextAlignment="Right" Margin="0,4,0,8"/>
                <Button x:Name="OrdersButton" Content="Заказы" Width="120" Height="34" Margin="0,0,0,8"
                    Background="#7FFF00" Click="Orders_Click"/>
                <Button Content="Выход" Width="120" Height="34" Background="#00FA9A" Click="Logout_Click"/>
            </StackPanel>
        </Grid>
    </Grid>

```

```

<StackPanel x:Name="FilterPanel" Grid.Row="1" Orientation="Horizontal" Margin="0,0,0,8">
    <TextBlock Text="Поиск:" VerticalAlignment="Center" Margin="0,0,6,0"/>
    <TextBox x:Name="SearchTextBox" Width="320" Margin="0,0,16,0" TextChanged="SearchTextBox_TextChanged"/>

    <TextBlock Text="Поставщик:" VerticalAlignment="Center" Margin="0,0,6,0"/>
    <ComboBox x:Name="SupplierComboBox" Width="260" Margin="0,0,16,0"
        SelectionChanged="SupplierComboBox_SelectionChanged"/>

    <TextBlock Text="Сортировка:" VerticalAlignment="Center" Margin="0,0,6,0"/>
    <ComboBox x:Name="SortComboBox" Width="250" SelectionChanged="SortComboBox_SelectionChanged">
        <ComboBoxItem Content="Без сортировки" IsSelected="True"/>
        <ComboBoxItem Content="Остаток (по возрастанию)"/>
        <ComboBoxItem Content="Остаток (по убыванию)"/>
    </ComboBox>
</StackPanel>

<StackPanel x:Name="AdminPanel" Grid.Row="2" Orientation="Horizontal" Margin="0,0,0,8">
    <Button Content="Добавить товар" Width="170" Height="34" Margin="0,0,10,0" Background="#00FA9A"
        Click="Add_Click"/>
    <Button Content="Редактировать товар" Width="190" Height="34" Margin="0,0,10,0" Background="#7FFF00"
        Click="Edit_Click"/>
    <Button Content="Удалить товар" Width="160" Height="34" Background="#7FFF00" Click="Delete_Click"/>
</StackPanel>

<DataGrid Grid.Row="3"
    x:Name="ProductsGrid"
    AutoGenerateColumns="False"
    IsReadOnly="True"
    HeadersVisibility="Column"
    GridLinesVisibility="Horizontal"
    RowHeight="68"
    MouseDoubleClick="ProductsGrid_MouseDoubleClick">
    <DataGrid.RowStyle>
        <Style TargetType="DataGridRow">
            <Setter Property="Background" Value="{Binding RowBrush}"/>
            <Setter Property="Foreground" Value="{Binding RowForeground}"/>
        </Style>
    </DataGrid.RowStyle>

    <DataGrid.Columns>
        <DataGridTemplateColumn Header="Фото" Width="100">
            <DataGridTemplateColumn.CellTemplate>
                <DataTemplate>
                    <Image Source="{Binding PhotoImage}" Width="90" Height="60" Stretch="Uniform"/>
                </DataTemplate>
            </DataGridTemplateColumn.CellTemplate>
        </DataGridTemplateColumn>

        <DataGridTextColumn Header="Артикул" Binding="{Binding Article}" Width="120"/>
        <DataGridTextColumn Header="Наименование" Binding="{Binding Name}" Width="170"/>
        <DataGridTextColumn Header="Категория" Binding="{Binding Category}" Width="150"/>
        <DataGridTextColumn Header="Описание" Binding="{Binding Description}" Width="240"/>
        <DataGridTextColumn Header="Производитель" Binding="{Binding Manufacturer}" Width="140"/>
        <DataGridTextColumn Header="Поставщик" Binding="{Binding Supplier}" Width="140"/>

        <DataGridTemplateColumn Header="Цена" Width="120">
            <DataGridTemplateColumn.CellTemplate>
                <DataTemplate>
                    <TextBlock Text="{Binding Price, StringFormat={}{0:N2}}">
                        <TextBlock.Style>
                            <Style TargetType="TextBlock">
                                <Setter Property="Foreground" Value="Black"/>
                                <Setter Property="TextDecorations" Value="{x:Null}"/>
                                <Style.Triggers>
                                    <DataTrigger Binding="{Binding HasDiscount}" Value="True">
                                        <Setter Property="Foreground" Value="Red"/>
                                        <Setter Property="TextDecorations" Value="Strikethrough"/>
                                    </DataTrigger>
                                </Style.Triggers>
                            </Style>
                        </TextBlock.Style>
                    </TextBlock>
                </DataTemplate>
            </DataGridTemplateColumn.CellTemplate>
        </DataGridTemplateColumn>

        <DataGridTextColumn Header="Цена со скидкой" Binding="{Binding FinalPrice, StringFormat={}{0:N2}}">
            <Width="130"/>
        </DataGridTextColumn>
        <DataGridTextColumn Header="Ед." Binding="{Binding UnitName}" Width="80"/>
        <DataGridTextColumn Header="Остаток" Binding="{Binding StockQuantity}" Width="90"/>
        <DataGridTextColumn Header="Скидка %" Binding="{Binding DiscountPercent}" Width="95"/>
    </DataGrid.Columns>
</DataGrid>
</Grid>
</Window>

```

Replace MainWindow.xaml.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows;

namespace ShoeStore2026PUApp

```

```

{
    public partial class MainWindow : Window
    {
        private readonly UserInfo _currentUser;
        private List<ProductRow> _allProducts = new List<ProductRow>();
        private ProductFormWindow _openedEditor;
        private bool _uiReady;

        public MainWindow(UserInfo user)
        {
            InitializeComponent();
            _currentUser = user ?? new UserInfo
            {
                UserId = null,
                FullName = "Гость",
                RoleName = "Гость"
            };

            if (string.IsNullOrEmpty(_currentUser.RoleName))
                _currentUser.RoleName = "Гость";
            if (string.IsNullOrEmpty(_currentUser.FullName))
                _currentUser.FullName = "Гость";

            RoleTextBlock.Text = "Роль: " + GetRoleCaption(_currentUser.RoleName);
            UserTextBlock.Text = "Пользователь: " + _currentUser.FullName;
            OrdersButton.Visibility = IsManagerOrAdmin() ? Visibility.Visible : Visibility.Collapsed;

            var advancedAllowed = IsManagerOrAdmin();
            FilterPanel.Visibility = advancedAllowed ? Visibility.Visible : Visibility.Collapsed;
            AdminPanel.Visibility = IsAdmin() ? Visibility.Visible : Visibility.Collapsed;

            if (advancedAllowed)
                LoadSuppliersForFilter();

            _uiReady = true;
            LoadProducts();
        }

        private void LoadProducts()
        {
            try
            {
                _allProducts = DataService.GetProducts();
                ApplyFiltersAndSorting();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message, "Ошибка БД", MessageBoxButton.OK, MessageBoxImage.Error);
            }
        }

        private void LoadSuppliersForFilter()
        {
            SupplierComboBox.Items.Clear();
            SupplierComboBox.Items.Add("Все поставщики");
            foreach (var s in DataService.GetSupplierNames())
                SupplierComboBox.Items.Add(s);
            SupplierComboBox.SelectedIndex = 0;
        }

        private void ApplyFiltersAndSorting()
        {
            if (!_uiReady || ProductsGrid == null)
                return;

            IEnumerable<ProductRow> query = _allProducts;

            if (IsManagerOrAdmin())
            {
                var search = (SearchTextBox.Text ?? "").Trim().ToLowerInvariant();
                var supplier = SupplierComboBox.SelectedItem?.ToString() ?? "Все поставщики";
                var sort = (SortComboBox.SelectedItem as System.Windows.Controls.ComboBoxItem)?.Content?.ToString() ??
                    "Без сортировки";

                if (supplier != "Все поставщики")
                    query = query.Where(x => string.Equals(x.Supplier, supplier, StringComparison.OrdinalIgnoreCase));

                if (!string.IsNullOrEmpty(search))
                {
                    query = query.Where(x =>
                        (x.Article ?? "").ToLowerInvariant().Contains(search) ||
                        (x.Name ?? "").ToLowerInvariant().Contains(search) ||
                        (x.Category ?? "").ToLowerInvariant().Contains(search) ||
                        (x.Description ?? "").ToLowerInvariant().Contains(search) ||
                        (x.Manufacturer ?? "").ToLowerInvariant().Contains(search) ||
                        (x.Supplier ?? "").ToLowerInvariant().Contains(search) ||
                        (x.UnitName ?? "").ToLowerInvariant().Contains(search));
                }

                if (sort == "Остаток (по возрастанию)")
                    query = query.OrderBy(x => x.StockQuantity);
                else if (sort == "Остаток (по убыванию)")
                    query = query.OrderByDescending(x => x.StockQuantity);
            }

            ProductsGrid.ItemsSource = query.ToList();
        }
    }
}

```

```

    }

    private void SearchTextBox_TextChanged(object sender, System.Windows.Controls.TextChangedEventArgs e)
    {
        if (!_uiReady) return;
        ApplyFiltersAndSorting();
    }

    private void SupplierComboBox_SelectionChanged(object sender,
    System.Windows.Controls.SelectionChangedEventArgs e)
    {
        if (!_uiReady) return;
        ApplyFiltersAndSorting();
    }

    private void SortComboBox_SelectionChanged(object sender, System.Windows.Controls.SelectionChangedEventArgs e)
    {
        if (!_uiReady) return;
        ApplyFiltersAndSorting();
    }

    private void Add_Click(object sender, RoutedEventArgs e)
    {
        OpenEditor(null);
    }

    private void Edit_Click(object sender, RoutedEventArgs e)
    {
        var selected = ProductsGrid.SelectedItem as ProductRow;
        if (selected == null)
        {
            MessageBox.Show("Выберите товар для редактирования.", "Информация", MessageBoxButton.OK,
            MessageBoxImage.Information);
            return;
        }
        OpenEditor(selected.ProductId);
    }

    private void ProductsGrid_MouseDoubleClick(object sender, System.Windows.Input.MouseButtonEventArgs e)
    {
        if (!IsAdmin()) return;
        var selected = ProductsGrid.SelectedItem as ProductRow;
        if (selected != null)
            OpenEditor(selected.ProductId);
    }

    private void OpenEditor(int? productId)
    {
        if (!IsAdmin())
        {
            MessageBox.Show("Добавлять и редактировать товары может только администратор.",
            "Доступ запрещен", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        // Блокируем открытие нескольких окон редактирования одновременно.
        if (_openedEditor != null && _openedEditor.IsVisible)
        {
            MessageBox.Show("Окно редактирования уже открыто.", "Информация", MessageBoxButton.OK,
            MessageBoxImage.Information);
            return;
        }

        _openedEditor = new ProductFormWindow(productId) { Owner = this };
        var result = _openedEditor.ShowDialog();
        _openedEditor = null;

        if (result == true)
        {
            LoadSuppliersForFilter();
            LoadProducts();
        }
    }

    private void Delete_Click(object sender, RoutedEventArgs e)
    {
        if (!IsAdmin())
        {
            MessageBox.Show("Удалять товары может только администратор.",
            "Доступ запрещен", MessageBoxButton.OK, MessageBoxImage.Warning);
            return;
        }

        var selected = ProductsGrid.SelectedItem as ProductRow;
        if (selected == null)
        {
            MessageBox.Show("Выберите товар для удаления.", "Информация", MessageBoxButton.OK,
            MessageBoxImage.Information);
            return;
        }

        var confirm = MessageBox.Show("Удалить выбранный товар?", "Подтверждение",
        MessageBoxButton.YesNo, MessageBoxImage.Question);
        if (confirm != MessageBoxResult.Yes) return;

        try
    
```

```

        {
            if (DataService.ProductExistsInOrders(selected.ProductId))
            {
                MessageBox.Show("Товар присутствует в заказе. Удаление невозможно.",
                    "Удаление запрещено", MessageBoxButton.OK, MessageBoxImage.Warning);
                return;
            }

            DataService.DeleteProduct(selected.ProductId);
            LoadSuppliersForFilter();
            LoadProducts();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Ошибка удаления", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    private void Orders_Click(object sender, RoutedEventArgs e)
    {
        if (!IsManagerOrAdmin()) return;
        var wnd = new OrdersWindow(_currentUser) { Owner = this };
        wnd.ShowDialog();
    }

    private void Logout_Click(object sender, RoutedEventArgs e)
    {
        var loginWindow = new LoginWindow();
        loginWindow.Show();
        Close();
    }

    private bool IsAdmin()
    {
        return string.Equals(_currentUser?.RoleName, "Администратор", StringComparison.OrdinalIgnoreCase);
    }

    private bool IsManagerOrAdmin()
    {
        return string.Equals(_currentUser?.RoleName, "Менеджер", StringComparison.OrdinalIgnoreCase) ||
            string.Equals(_currentUser?.RoleName, "Администратор", StringComparison.OrdinalIgnoreCase);
    }

    private string GetRoleCaption(string role)
    {
        if (role == "Авторизированный клиент") return "Клиент";
        if (role == "Администратор") return "Администратор";
        if (role == "Менеджер") return "Менеджер";
        return "Гость";
    }
}
}

```

Step 16 Execute приложение и check модули 2-4

Check роли, список товаров, CRUD товаров и работу заказов.

Commands/code

Execute:

1. Отладка -> Начать отладку (F5).

Check:

- enter как гость и enter под пользователем из users;
- ФИО в правом верхнем углу;
- Выход возвращает в окно входа;
- у гостя/клиента: только просмотр товаров;
- у менеджера: поиск, фильтр, сортировка и просмотр заказов;
- у администратора: CRUD товаров, CRUD заказов.

Check requirements интерфейса:

- если discount_percent > 15, строка #2E8B57;
- если скидка > 0, базовая цена зачеркнута красным;
- если stock_quantity = 0, строка голубая;
- при отсутствии фото используется picture.png;

- при выборе нового фото (до 300x200) оно копируется в папку приложения, старое удаляется.

Step 16.1 Commit локальный git-коммит (модули 2-4)

Make локальный коммит после реализации функционала модулей 2-4.

Commands/code

```
cd C:\shoe_store_2026_pu_csharp\ShoeStore2026PUApp
git init
git add .
git commit -m "Реализованы модули 2-4 (C#, Вариант 1 2026)"
```

Step 17 Prepare документы SQL/ER и final набор файлов

Collect обязательные документы, экспорт БД, итоговые файлы проекта и commit локальный коммит.

Commands/code

1. Open draw.io (diagrams.net).
2. Create new схему: File -> New -> Blank Diagram.
3. Set A4 (Меню Файл -> Параметры страницы).
4. Collect блок-схему по ГОСТ 19.701-90:
 - Блок начала/конца (овал): Начало.
 - Процесс (прямоугольник): Открыть окно входа.
 - Ввод/вывод (параллелограмм): Ввод логина и пароля / выбор "Войти как гость".
 - Решение (ромб): Гость?.
 - Процесс: Показать список товаров (роль Гость) (ветка Да).
 - Решение: Логин/пароль верны? (ветка Нет -> Сообщение об ошибке -> возврат к вводу).
 - Процесс: Определить роль (клиент/менеджер/администратор) (ветка Да).
 - Процесс: Показать список товаров.
 - Процесс: Поиск/фильтр/сортировка.
 - Решение: Роль позволяет редактирование?.
 - Процесс: CRUD товаров и заказов (для менеджера/администратора).
 - Блок начала/конца (овал): Выход.
5. Connect блоки стрелками по потоку выполнения.
6. Save исходник схемы:
 - C:\shoe_store_2026_pu_csharp\docs\algorithm_gost.drawio
7. Export PDF:
 - Файл -> Экспортировать как -> PDF.
 - путь: C:\shoe_store_2026_pu_csharp\docs\algorithm_gost.pdf
3. Create:
 - C:\shoe_store_2026_pu_csharp\docs\report_screenshots.docx
4. Add скриншоты:
 - окно входа;
 - вход как гость;
 - вход под менеджером;
 - вход под администратором;

- поиск/фильтрация/сортировка;
- добавление/редактирование/удаление товара;
- окно заказов;
- добавление/редактирование/удаление заказа.

5. В phpMyAdmin export БД shoe2026_pu в SQL:

- C:\shoe_store_2026_pu_csharp\sql\shoe2026_pu.sql

6. В phpMyAdmin -> Ещё -> Дизайнер export ER-диаграмму:

- C:\shoe_store_2026_pu_csharp\sql\shoe2026_pu_er.pdf

7. В Visual Studio switch конфигурацию на Release.

8. Меню Сборка -> Собрать решение.

9. Check итоговую папку:

- C:\shoe_store_2026_pu_csharp\ShoeStore2026PUApp\ShoeStore2026PUApp\bin\Release\

For WPF (.NET Framework 4.8) worked результатом usually является folder bin\Release целиком, а не один .exe.

Commit итог в локальном git-репозитории:

```
cd C:\shoe_store_2026_pu_csharp\ShoeStore2026PUApp
git add .
git status
git commit -m "Финальная версия проекта (C#, Вариант 1 2026)"
```